School of Computer Science Engineering and Information Systems
B.Tech Information Technology
BCSE355L : Cloud Architecture Design
FALL 2026-2027
Course Project -Review II

TEAM NAME : **Nebula Knight**
Team Member(s) with Reg # and Name : **Moorthy Chetan; 24BIT0246; Mobile: 9441522812; moorthychetan06@gmail.com**
Project Title : **CloudPulse: Autonomous Cloud Reliability and Self-Healing Control Plane Using Real AWS Multi-Service Architecture**

1. Introduction

This section gives the background, the problem addressed, the objectives and the related literature.

1.1 Background

Cloud computing is now the default delivery platform for software, so availability and fast fault recovery are core quality requirements rather than optional extras. Conventional monitoring stacks are very good at **detecting** a problem (a metric crosses a threshold, an alarm fires, an engineer is paged) but the remaining steps of the reliability cycle – diagnosing, choosing a remedy, executing it and recording what happened – usually still depend on a human operator. Every minute spent in that manual gap adds to the Mean Time To Recover (MTTR) and to operational cost.

Serverless services change the economics of closing that gap. Functions-as-a-Service (AWS Lambda), managed event routing (Amazon EventBridge), managed metrics and alarms (Amazon CloudWatch), a managed key-value store (Amazon DynamoDB) and managed messaging (Amazon SNS/SQS) allow a complete detect-decide-act-record pipeline to be assembled without a single always-on server. Because the control plane scales to zero when idle, it also stays cheap, which matters for a student-scale project.

CloudPulse applies this idea to an educational setting. It is an event-driven, closed-loop reliability and self-healing simulator built on AWS serverless services. A controlled failure is injected into a resource; CloudWatch detects the degraded metric; EventBridge routes the event; a Recovery Lambda selects and executes a deterministic remediation strategy; the incident and its full audit trail are stored in DynamoDB; SNS notifies the operator; and a React dashboard shows the whole lifecycle live. The final release (v2) additionally allows the same recovery engine to act on real EC2, Auto Scaling, Application Load Balancer and

RDS resources, but only for resources that the operator has explicitly allow-listed (see Section 2.3 and 3.1).

1.2 Problem Statement

Traditional cloud monitoring primarily detects failures and notifies administrators. A student-scale system rarely demonstrates the **complete** cycle from controlled failure generation, through automated detection and recovery, to incident auditing, reliability analytics and notification. The problem addressed by this project is:

> *How can a cloud system automatically detect a failure, execute an appropriate recovery strategy exactly once, record the incident lifecycle, quantify reliability, and notify users – using a scalable, event-driven, low-cost serverless architecture and without putting real production infrastructure at risk by default?*

The sub-problems that follow from this statement are listed with the objective that answers each of them and the place in the repository where the answer can be verified:

#	Objective	How it is met in CloudPulse	Evidence in repository
O1	Generate repeatable failures safely	Five deterministic scenarios (FS-01 to FS-05) with fixed fault profiles; default SIMULATION mode touches only DynamoDB state and custom metrics	`backend/app/models/resource.py`, `services/simulation_service.py`
O2	Detect failures automatically	12 CloudWatch alarms (3 per resource), 60 s period, 2 evaluation periods, plus a direct EventBridge fast path	`infrastructure/template.yaml`
O3	Recover without a human	Recovery Lambda with a strategy dispatcher, 8-state machine and at most 3 attempts	`lambda/recovery/handler.py`, `strategies.py`
O4	Never recover twice for one event	Two-layer idempotency: pre-read gate and DynamoDB conditional write	`lambda/recovery/handler.py`
O5	Keep an auditable record	Incident document with embedded recovery-action list, timestamps and notification status	`backend/app/models/incident.py`
O6	Quantify reliability	MTTR, MTBF, success/failure rate, detection time, incident frequency and health distribution computed from stored records	`services/reliability_calculator.py`
O7	Notify and visualise	SNS lifecycle messages; React/TypeScript dashboard with adaptive polling	`services/notification_service.py`, `frontend/src`
O8	Stay cheap and reproducible	Pay-per-request DynamoDB, arm64 Lambda, one SAM template, CI pipeline	`infrastructure/template.yaml`, `.github/workflows/ci.yml`

1.3 Literature Survey

Ten journal papers (single-member team: 10 per student) were reviewed. Each is listed with the part of the CloudPulse design it informs; the rest of this document concentrates on the implementation in the repository.

Ref	Citation	Relevance to CloudPulse
1	J. Scheuner and P. Leitner, “Function-as-a-Service performance evaluation: A multivocal literature review,” Journal of Systems and Software, vol. 170, 2020.	Justifies AWS Lambda as the execution substrate and the care taken to describe recovery time as a recorded, not benchmarked, value.
2	P. Leitner, E. Wittern, J. Spillner and W. Hummer, “A mixed-	Event-driven background processing and the

	method empirical study of Function-as-a-Service software development in industrial practice," Journal of Systems and Software, vol. 149, pp. 340–359, 2019.	monitoring/debugging pain points motivate the recovery design, structured logging with correlation IDs and the large automated test suite.
3	V. Yussupov et al., "FaaSten your decisions: A classification framework and technology review of Function-as-a-Service platforms," Journal of Systems and Software, vol. 175, 2021.	Vendor lock-in is an explicit trade-off; provider calls are confined to `aws/clients.py` and the strategy layer.
4	A. Mampage, S. Karunasekera and R. Buyya, "A holistic view on resource management in serverless computing environments: Taxonomy and future directions," ACM Computing Surveys, vol. 54, no. 11s, 2022.	Places SCALE_OUT and FAILOVER in the resource-management taxonomy; rule-based remediation is the baseline.
5	G. A. S. Cassel et al., "Serverless computing for Internet of Things: A systematic literature review," Future Generation Computer Systems, vol. 128, 2022.	The simulator's heartbeat telemetry mirrors a stream of metrics evaluated by threshold rules.
6	M. Enes, R. R. Expósito and J. Touriño, "Real-time resource scaling platform for Big Data workloads on serverless environments," Future Generation Computer Systems, 2020.	Metric-driven automatic scaling; the SCALE_OUT strategy is a simpler deterministic counterpart.
7	M. Malawski, A. Gajek, A. Zima, B. Balis and K. Figiela, "Serverless execution of scientific workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions," Future Generation Computer Systems, 2020.	Lambda limits (timeouts, statelessness) shape the design: handlers stay under 60 s and keep all state in DynamoDB.
8	S. Ristov, S. Pedratscher and T. Fahringer, "AFCL: An abstract function choreography language for serverless workflow specification," Future Generation Computer Systems, vol. 114, pp. 368–382, 2021.	Informs the explicit, declared workflow detect, route, heal, record, notify.
9	A. Brogi, J. Carrasco, F. Durán, E. Pimentel and J. Soldani, "Self-healing trans-cloud applications," Computing, vol. 104, no. 4, pp. 809–833, 2022.	Closest concept: failure-type to action rules with bounded retries and an escalation path.
10	D. Cotroneo, L. De Simone, P. Liguori and R. Natella, "Enhancing the analysis of software failures in cloud computing systems with deep learning," Journal of Systems and Software, 2021, doi:10.1016/j.jss.2021.111043.	Supports deliberate fault injection as a way to study failures; CloudPulse injects faults but uses transparent rules.

Taken together, the papers show that serverless platforms suit event-driven back-ends, that resource management is the most studied operational problem, and that self-healing and fault injection are active topics. A small, fully reproducible, end-to-end implementation that connects fault injection, metric-based detection, exactly-once automated remediation and reliability analytics using only managed services is what CloudPulse provides as an academic reference system.

2. Overview and Planning

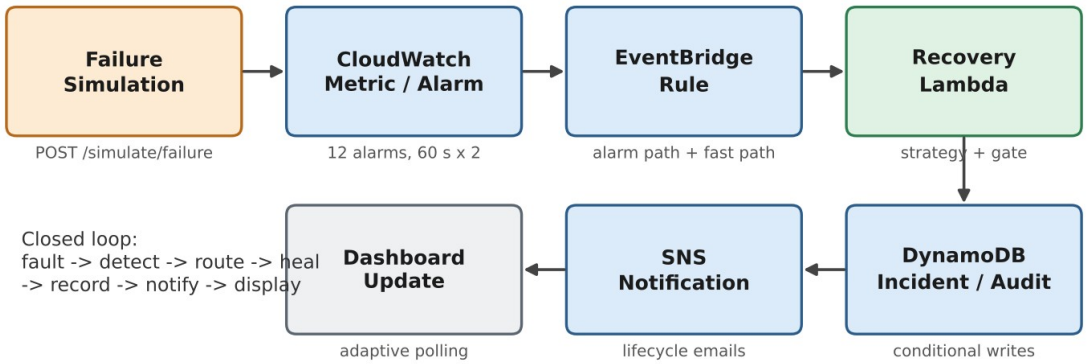
This section describes the proposed system, the project plan, the challenges, the assumptions and the architecture, hardware and software requirements.

2.1 Proposed System Overview

CloudPulse is a serverless cloud reliability and self-healing simulator. Virtual resources generate controlled failures; CloudWatch detects degraded metrics; EventBridge routes events; Lambda performs recovery; DynamoDB records incidents; SNS sends notifications; and a React dashboard presents live state. The core flow is:

Failure Simulation → CloudWatch Metric/Alarm → EventBridge Rule → Recovery Lambda → DynamoDB Incident/Audit → SNS Notification → Dashboard Update

Figure 1: CloudPulse closed-loop self-healing flow



Two routes lead from a fault to the Recovery Lambda. On the **alarm path** the injected metric values are published to CloudWatch, an alarm enters the ALARM state after two consecutive one-minute breaches and EventBridge forwards the state-change event. On the **fast path** the API additionally emits a `FailureInjected` event (source `cloudpulse.simulator`) directly to EventBridge so a live demonstration does not need to wait two minutes. Both routes land in the same handler and are protected by the same idempotency gate, so a fault that arrives by both routes is healed once.

The four managed resources

The simulator manages four virtual resources, created by `scripts/seed_data.py`; the table shows their seed metrics and the alarms deployed for each.

Resource ID	Type	Nominal CPU / Mem / Storage / Latency	Alarms deployed
VM-001	VM	25 % / 30 % / 20 % / 15 ms	HIGH_CPU, NETWORK_LATENCY, SERVICE_HEALTH
API-001	API	15 % / 20 % / 10 % / 20 ms	NETWORK_LATENCY, HIGH_CPU, SERVICE_HEALTH
DB-001	DB	40 % / 55 % / 60 % / 10 ms	HIGH_CPU, NETWORK_LATENCY, SERVICE_HEALTH
STORAGE-001	STORAGE	see `tests/fixtures/seed_resources.json`	STORAGE_EXHAUSTION, NETWORK_LATENCY, SERVICE_HEALTH

The table lists the seed values in the repository fixtures; the in-code model default for a new resource is 25 / 30 / 20 / 15. Only STORAGE-001 has a storage-utilisation alarm, so FS-03 is detected only on that resource.

Real AWS fleet (HYBRID / REAL_AWS deployments)

In ap-south-1 the v2 demonstration fleet comprises one EC2 t3.micro (Amazon Linux 2023), an Auto Scaling Group (1–3), an Application Load Balancer with primary and standby target groups, one RDS PostgreSQL 16 db.t4g.micro instance and an SSM-managed node. Identifiers below are as reported in the repository README (owner-reported; not independently checked here). Public IP addresses and the ALB DNS name are deliberately left out of the body.

Resource	AWS service	Specification	Real SDK call used by the recovery engine
`i-083e8c6b1fc9ca0b7`	EC2	t3.micro, Amazon Linux 2023,	`ec2.reboot_instances`

		ap-south-1a	
asg-cloudpulse-fleet	Auto Scaling	min 1, max 3, desired 2	`describe_auto_scaling_groups`, `set_desired_capacity(current + 1, HonorCooldown=True)`
alb-cloudpulse-gw	Elastic Load Balancing v2	Target groups `tg-primary`, `tg-standby`	`modify_listener` (default action to the standby target group)
rds-cloudpulse-db	RDS	PostgreSQL 16, db.t4g.micro, port 5432	`reboot_db_instance` (ForceFailover first, plain reboot on error)
SSM managed node (profile `cloudpulse-ec2-ssm-profile`)	Systems Manager	Run Command, `AWS-RunShellScript`	`ssm.send_command` (read-only diagnostics)

"Five real AWS services" in the repository history refers to this real target fleet (EC2, Auto Scaling, Elastic Load Balancing v2, RDS and SSM). The serverless control plane itself uses Lambda, API Gateway, DynamoDB, CloudWatch, EventBridge, SNS, SQS, S3 and IAM. Scenario-to-API mapping is in Section 3.2.5.

Release v2: optional real-AWS data plane

The final release adds a hybrid execution mode. The control plane (API Gateway, Lambda, EventBridge, CloudWatch, DynamoDB, SNS, SQS, S3) is unchanged and always serverless. The recovery strategies can, however, call real AWS APIs for resources that are registered in the resources table **and** listed in the `REAL_AWS_ALLOWLIST` deployment parameter. The fleet is listed above. Failures on these resources are still **injected** through the same API (logical fault); what becomes real is the remediation call (Section 3.2.5).

Project plan and milestones

The following timeline is reconstructed from the repository commit history (32 commits on `main` at the time of writing, per `git rev-list --count HEAD` reported by the project owner). Some commits are dated after the Review-II window; they are included because the document describes the final implementation.

Date	Milestone	Outcome
08 Sep 2026	Planning and foundation	Project constitution, architecture, ADRs, folder structure; SAM template and safe management scripts
09 Sep 2026	Core build	DynamoDB data layer and simulation engine; CloudWatch/EventBridge monitoring; recovery Lambda and strategies; incident lifecycle, SNS and reliability metrics; React dashboard and API integration
09 Sep 2026	Audit and documentation	Architecture audit (critical/high findings fixed); final technical audit; Review-2 documentation package
10–11 Sep 2026	Deployment hardening	Deploy/seed script fixes; API Gateway stage-prefix routing fix; architecture presentation
07 Oct 2026	Repository quality	MIT licence, CI workflow (backend and frontend), README quick-start
24 Oct 2026	Release v2	Hybrid real-AWS remediation behind allow-list; reliability/recovery/safety audit fixes; EventBridge pattern alignment; real-fleet CLI; documentation sync

2.2 Challenges

The main technical challenges and the way the repository handles each are listed below.

Challenge	Why it is hard	How CloudPulse handles it

Reliable event correlation	An alarm name is the only link between a CloudWatch event and a resource	Naming convention `cloudpulse-{ResourceId}-{FailureType}`; suffix-first parsing so IDs with hyphens (STORAGE-001) parse correctly
Idempotent recovery	EventBridge is at-least-once; the alarm path and fast path can both fire	Pre-read state gate plus conditional DynamoDB update; duplicate events exit without a second action
Deterministic scenarios	Random metrics make tests and demos unrepeatable	Fixed fault profiles per scenario; random-walk jitter only in the heartbeat simulator
Safe AWS usage	Real remediation could damage real infrastructure	SIMULATION default; real calls need non-simulation mode and explicit allow-list; allowed resources must be registered
Observability	Distributed asynchronous steps are hard to follow	Structured JSON logs with correlation ID, incident ID and trace stage
False positives	A single metric spike should not trigger recovery	Alarms need 2 consecutive 60 s breaches; missing data treated as not breaching
Partial failure	CloudWatch, SNS or EventBridge may fail while recovery succeeds	Monitoring and notification calls are non-fatal; failures are logged; recovery result is still persisted
Retry storms	A failing strategy could be retried forever	MAX_RECOVERY_ATTEMPTS = 3, then MANUAL_INTERVENTION_REQUIRED and incident ESCALATED; SQS dead-letter queue for Lambda failures
CORS and exposure	Browser calls to API Gateway need CORS; open APIs are risky	CORS origin is a template parameter (default localhost:5173); S3 bucket private by default
Cost control	Idle infrastructure costs money	Pay-per-request DynamoDB, arm64 Lambda, 7-day log retention, teardown CLI for the real fleet

2.3 Assumptions

The design rests on the following assumptions.

- Failures are **injected logically**: the fault is a state mutation in DynamoDB plus custom CloudWatch metric values. Injection never calls EC2, RDS or VPC APIs.
- AWS control-plane services (Lambda, API Gateway, DynamoDB, CloudWatch, EventBridge, SNS, SQS, S3, IAM) are real and billed normally.
- The default deployment runs in `SIMULATION` mode. Real remediation exists only when the operator deploys with `ExecutionMode=HYBRID` or `REAL_AWS` **and** lists specific resource IDs in `RealAwsAllowlist`; an empty allow-list means no real call is ever made.
- The workload is small and educational (four virtual resources, a handful of concurrent incidents).
- Scenarios and recovery strategies are predefined and deterministic; CloudPulse does not learn or predict.
- A single AWS account and region (ap-south-1) are used; the API has no end-user authentication (accepted for a demonstration, see Section 4.8).
- Strategy durations stored in code (2–6 s) are defaults used in SIMULATION. In the live HYBRID deployment, `duration_seconds` on an incident is computed from stored timestamps (`detected_at` to `resolved_at`). It measures the time until the recovery call returned, not the time until the service was verified healthy (see Section 4.9).

2.4 Architecture Specifications

The architecture is layered as follows; each layer maps to a directory or a template resource in the repository.

Layer	Technology	Role
Frontend	React 18.3, TypeScript 5.5, Vite 5, Tailwind CSS 3.4, Axios	Nine-section SRE console with adaptive polling (3 s active / 10 s idle, paused when the tab is hidden)
Ingress	Amazon API Gateway HTTP API (v2)	Routes `/{proxy+}`, `/` and `/health` to the API Lambda; CORS; JSON access logs
Application	FastAPI 0.115 + Mangum 0.17 on AWS Lambda (Python 3.12, arm64)	Resource, incident, metric and simulation endpoints
Simulation	Simulator Lambda, EventBridge schedule `rate(2 minutes)`	Heartbeat telemetry with jitter; metrics batched into `put_metric_data` calls
Monitoring	CloudWatch custom metrics (namespace `CloudPulse`) and 12 alarms	Detection
Event routing	Amazon EventBridge (default bus), two rules on the Recovery Lambda	Alarm path and fast path
Recovery	Recovery Lambda (60 s timeout) with strategy dispatcher	Deterministic remediation, bounded attempts, audit
Data	DynamoDB: resources, incidents (GSI `ResourceIndex`), metrics tables, PAY_PER_REQUEST	State, incident records, reliability snapshots
Notification	Amazon SNS topic (optional e-mail subscription)	Failure-detected, recovery-started, recovered and failed messages
Failure handling	Amazon SQS dead-letter queue (1-day retention)	Captures failed Lambda invocations
Hosting and IaC	Amazon S3, AWS SAM / CloudFormation	Single template `infrastructure/template.yaml`; Lambda layer for shared code
Access control	Three least-privilege IAM roles (API, Simulator, Recovery)	See Section 4.8 for the scope of the Recovery role

Compute configuration

The three Lambda functions are defined in the SAM template with the settings below.

Function	Name pattern	Timeout	Trigger	Notes
API	cloudpulse-api-{env}	30 s	API Gateway (ANY /{proxy+}, ANY /, GET /health)	FastAPI via Mangum, `lifespan="off"`
Simulator	cloudpulse-simulator-{env}	60 s	EventBridge schedule rate(2 minutes)	Heartbeat metrics for all resources
Recovery	cloudpulse-recovery-{env}	60 s	EventBridge rules (alarm, FailureInjected)	DLQ: SQS; 256 MB, arm64, Python 3.12 (stack-wide defaults)

Alarm configuration

All 12 alarms use statistic Average, period 60 s, 2 evaluation periods and `TreatMissingData: notBreaching`. Thresholds: CPUUtilization ≥ 85 %, StorageUtilization ≥ 90 %, NetworkLatency ≥ 500 ms, ServiceHealth ≤ 50. Each alarm also lists the SNS topic as an alarm action.

Architecture decisions (ADRs)

Six architecture decision records in `docs/adr` document the main choices. All six still carry the status "Proposed" in the files although the implementation follows them; the one deviation is polling, where the ADR proposes 10–15 s but the final dashboard uses 3 s while active and 10 s while idle.

ADR	Decision	Main rationale
001 IaC tool	AWS SAM rather than CDK	Purpose-built for serverless; readable YAML for review; `sam local` for testing; lower learning curve
002 Recovery trigger	CloudWatch alarm → EventBridge rule → Recovery Lambda	Event-driven decoupling; built-in filtering, retry and DLQ; allows more consumers later
003 DynamoDB design	Multi-table instead of single-table	Easier to explain; resource and incident access patterns are separate; within Free Tier
004 Frontend hosting	S3 static website only, no CloudFront (phase 1)	Avoids CDN cost and invalidation complexity at demo scale; consequence: HTTP only without a custom domain
005 Real-time updates	Client-side polling, not WebSocket or SSE	API Gateway WebSocket needs connection management; HTTP API has no SSE; state changes are slow relative to polling
006 API serving	API Gateway HTTP API + Mangum, not Lambda Function URL	API Gateway is a course-required service; built-in CORS, throttling and access logs

Non-functional requirements

The quality attributes the design had to satisfy, and how the repository meets them, are summarised below.

Quality	Requirement	How it is addressed
Reliability	A failure must be healed once and the outcome recorded even if helpers fail	Idempotency gate; non-fatal CloudWatch/SNS calls; DLQ; bounded retries
Safety	No unintended change to real infrastructure	SIMULATION default; allow-list; read-only SSM diagnostics; teardown restricted to tagged resources
Performance	Dashboard reflects state within seconds during an incident	3 s adaptive polling; single-call batched metrics; Lambda timeouts 30–60 s
Scalability	Handle more resources or incidents without redesign	On-demand DynamoDB, stateless Lambdas; note that list endpoints use Scan (A-04), acceptable only at academic scale
Security	Least privilege and safe defaults	Three IAM roles, namespace-scoped metrics, private S3 default, localhost CORS default; open API accepted for demo
Cost	Stay inside a student budget	Pay-per-request, arm64, short log retention; v2 fleet optional with teardown command
Maintainability	Understandable, testable code	Layered backend, typed models, strategy pattern, 417 passing tests (422 collected, 5 skipped), CI
Portability	Reproducible deployment	Single SAM template, parameterised, scripts for deploy/seed/teardown

2.5 Hardware Requirements

The control plane needs no servers; hardware is needed only for development and, optionally, for the v2 real-AWS fleet.

Item	Minimum	Used for

Development laptop / desktop	Modern x86-64 or arm64 CPU, 4 GB RAM (8 GB recommended), 5 GB free disk	Editing, running pytest/vitest, `sam build`, local dashboard
Network	Stable internet connection	AWS CLI/SAM deployment, dashboard access
Peripherals	Keyboard, mouse, standard display	Dashboard demonstration
AWS control plane	No servers to provision	Runs entirely on managed serverless services
AWS data plane (v2, optional)	EC2 t3.micro, ASG (1–3), ALB, RDS PostgreSQL 16 db.t4g.micro	Only for the real-remediation demonstration; can be removed with `manage_real_aws.py teardown-fleet`

2.6 Software Requirements

Versions are those pinned in `backend/requirements.txt`, `frontend/package.json` and the CI workflow.

Software	Version	Purpose
Python	3.12	Lambda runtime, backend, tests
FastAPI / Pydantic / pydantic-settings	0.115.0 / 2.9.2 / 2.5.2	API framework, validation, configuration
Mangum	0.17.0	ASGI-to-Lambda adapter
boto3 / botocore	1.35.0	AWS SDK
pytest, moto	see `backend/requirements-dev.txt`	Unit and AWS-mock tests
Node.js	20 (CI)	Frontend build and tests
React / TypeScript / Vite / Tailwind	18.3.1 / 5.5.4 / 5.4.8 / 3.4.13 (declared ranges)	Dashboard
Vitest	2.1.2	Frontend tests
mypy (`backend/app`)	30 source files, 0 errors	Static typing (owner-run, README)
`tsc --noEmit` / `vite build`	0 type errors	Frontend type safety and production build
`sam validate --lint`	valid template	Infrastructure check
AWS SAM CLI, AWS CLI	current	Build, validate, deploy
Git / GitHub Actions	—	Source control and CI
AWS services	Lambda, API Gateway, DynamoDB, CloudWatch, EventBridge, SNS, SQS, S3, IAM, CloudFormation (+ EC2, Auto Scaling, ELBv2, RDS, SSM in v2)	Runtime platform

AWS services used and their roles

AWS service	Role in CloudPulse	Where it is defined or called
AWS Lambda	Runs the API (FastAPI + Mangum), the heartbeat Simulator and the Recovery function on Python 3.12 / arm64	`AWS::Serverless::Function` ×3; code in `lambda/`
Lambda layer	Packages the shared backend code and dependencies once for all three functions	`AWS::Serverless::LayerVersion` (`CloudPulseLayer`)
Amazon API Gateway (HTTP API)	Public REST entry point used by the dashboard and for failure injection	`AWS::Serverless::HttpApi`
Amazon DynamoDB	Resources, incidents (with audit trail) and metrics tables; conditional writes enforce the state machine	`AWS::DynamoDB::Table` ×3
Amazon CloudWatch	Custom metrics in namespace `CloudPulse` and 12 alarms that detect degradation	`AWS::CloudWatch::Alarm` ×12; `put_metric_data`
Amazon CloudWatch Logs	Structured logs for the three functions, retained 7 days	`AWS::Logs::LogGroup` ×4
Amazon EventBridge	Routes alarm state changes and the direct	Two rules in the template;

	`FailureInjected` event to the Recovery Lambda	`events.put_events`
Amazon SNS	Incident lifecycle notifications to the operator	`AWS::SNS::Topic` and `Subscription`
Amazon SQS	Dead-letter queue for failed recovery invocations	`AWS::SQS::Queue`
Amazon S3	Static website hosting for the React dashboard	`AWS::S3::Bucket`, `BucketPolicy`
AWS IAM	Three roles (API, Simulator, Recovery) with separate permissions	`AWS::IAM::Role` ×3
AWS CloudFormation / SAM	Builds and deploys the whole stack from one template with environment parameters	`infrastructure/template.yaml`, `sam build` / `sam deploy`
Amazon EC2	Worker instance rebooted by the SERVICE_RESTART and FAILOVER actions	`ec2.reboot_instances`
Amazon EC2 Auto Scaling	Fleet capacity raised by one on high CPU	`set_desired_capacity`
Elastic Load Balancing (ALB)	Listener switched from the primary to the standby target group on network degradation	`elbv2.modify_listener`
Amazon RDS (PostgreSQL 16)	Database rebooted, with forced failover requested first, on database downtime	`rds.reboot_db_instance`
AWS Systems Manager	Run Command executes read-only storage diagnostics on the instance	`ssm.send_command`
Amazon VPC	Private subnet that hosts the RDS instance (per the repository README)	Fleet created outside the template
AWS CloudTrail	Records the real API calls made by the Recovery role; used as evidence in Section 4.15	Account-level audit log

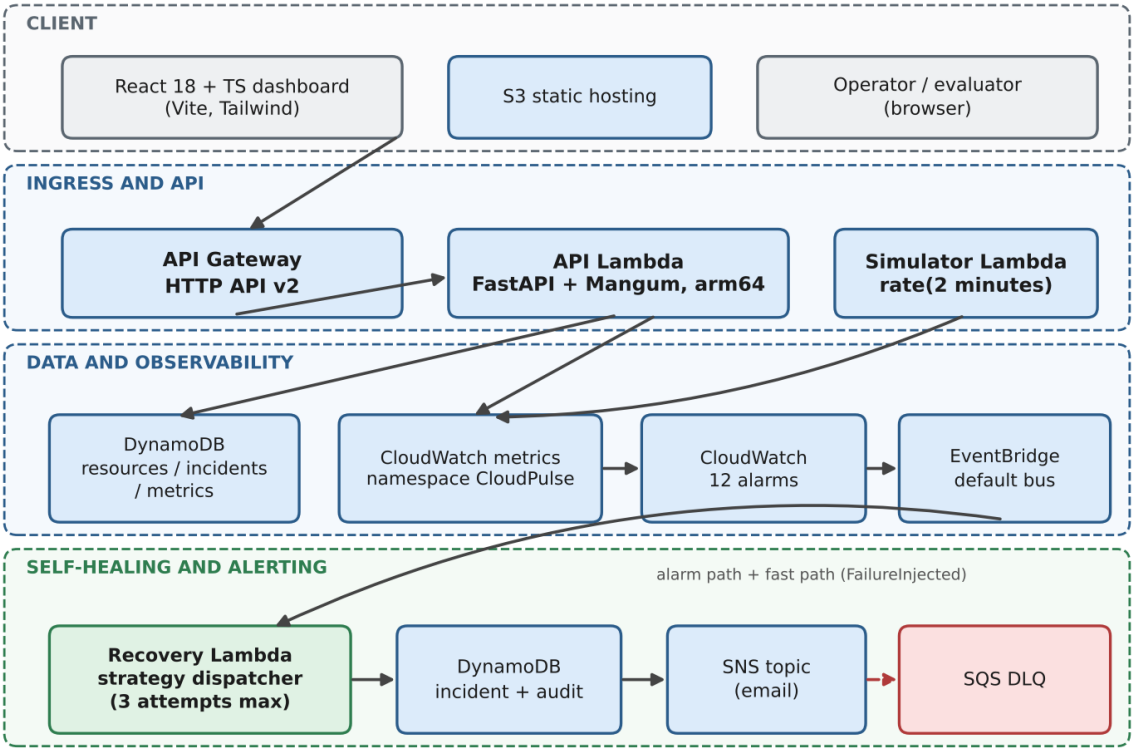
3. System Design

This section presents the high-level architecture and the low-level design of the data, the recovery engine, the API and the dashboard.

3.1 High-Level Design

The system is event-driven and serverless. The browser loads the React single-page application from S3 and calls API Gateway; the API Lambda reads and writes DynamoDB and publishes CloudWatch metrics and EventBridge events. A scheduled simulator keeps the resources' telemetry alive. CloudWatch alarms and the direct `FailureInjected` event both reach the Recovery Lambda, which updates DynamoDB, publishes SNS notifications and emits recovery metrics.

Figure 2: High-level architecture (AWS region ap-south-1)



End-to-end data flow

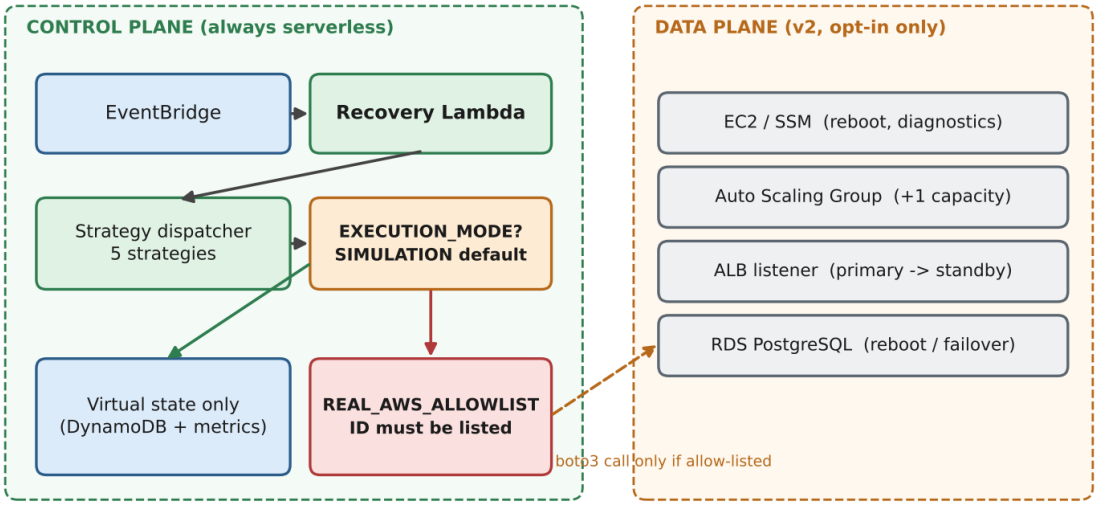
One injected failure passes through the ten steps below.

Step	Component	Action
1	Dashboard / API client	`POST /simulate/failure` with `resourceId`, `failureType`
2	API Lambda (SimulationService)	Validates the resource, sets state FAILURE_DETECTED, writes fault metrics, creates the incident (OPEN)
3	MonitoringService	Publishes five metrics for the resource to CloudWatch; emits `FailureInjected` to EventBridge (both non-fatal)
4	CloudWatch	After 2 consecutive 60 s breaches the matching alarm enters ALARM; state change goes to EventBridge
5	EventBridge	Rule 1: source `aws.cloudwatch`, alarm name prefix `cloudpulse-`, state ALARM. Rule 2: source `cloudpulse.simulator`, detail-type `FailureInjected`
6	Recovery Lambda	Parses the event; ignores SERVICE_HEALTH alarms; applies the idempotency gate
7	Strategy dispatcher	Selects the strategy for the failure type and executes it (simulated or allow-listed real action)
8	Recovery Lambda	Writes RECOVERED, resets metrics, closes the incident (RESOLVED) or escalates after 3 failed attempts
9	NotificationService	Publishes lifecycle messages to SNS, tracking `notified_transitions` to avoid duplicates
10	Dashboard	Adaptive polling shows the new state, incident detail and updated reliability metrics

Control plane and data plane

CloudPulse separates the always-on-demand serverless **control plane** from the **data plane** it protects. In the default deployment the data plane is virtual (DynamoDB records). In the hybrid deployment the same recovery engine can act on real resources, but only through a two-key gate: the execution mode must not be SIMULATION, and the resource ID must appear in the allow-list.

Figure 4: Serverless control plane and optional real-AWS data plane (v2)



Deployment view

Deployment is driven by the SAM template and the scripts in `scripts/`:

- `sam build` packages three Lambda functions (arm64) and a shared layer built from `backend/requirements.txt`; `sam deploy` creates the stack with parameters `Environment`, `NotificationEmail`, `CorsAllowedOrigins`, `CloudWatchNamespace`, `LogLevel`, `ExecutionMode`, `RealAwsAllowlist` and `EnablePublicFrontendBucket`.
- Stack outputs expose the API endpoint, the frontend bucket name and website URL, the three table names, the SNS topic ARN and the DLQ ARN.
- The frontend is built with Vite and uploaded to the S3 bucket by `scripts/deploy_frontend.sh`; the bucket is private unless `EnablePublicFrontendBucket=true`.

3.2 Low-Level Design

This section describes the data structures and algorithms behind the architecture: scenarios, states, tables, the recovery handler, the strategy layer, health scoring, the API and the dashboard.

3.2.1 Failure scenarios and recovery mapping

Five deterministic scenarios are implemented. The fault profile is the set of metric values written by `FAILURE_METRIC_TARGETS` in the resource model; these are the values actually used by the engine.

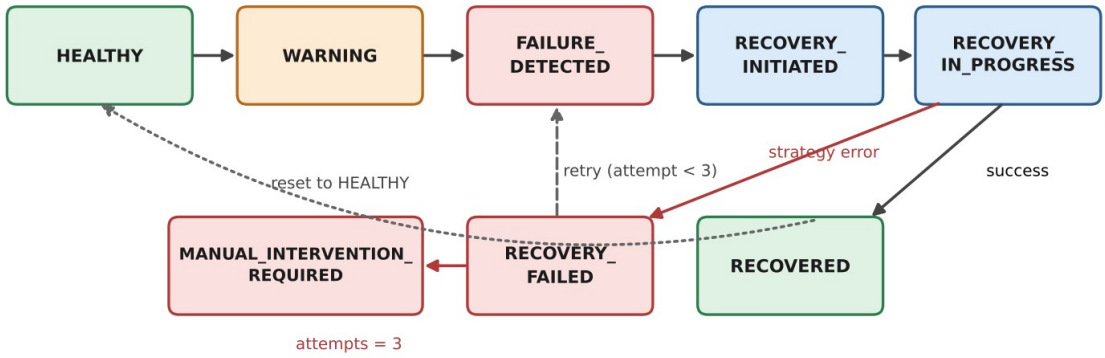
ID	Failure type	Fault profile (CPU / Mem / Storage / Latency)	Detecting signal	Recovery action
FS-01	HIGH_CPU	92 % / 60 % / 20 % / 50 ms	CPUUtilization ≥ 85	SCALE_OUT
FS-02	SERVICE_FAILURE	15 % / 20 % / 10 % / 980 ms	NetworkLatency ≥ 500	SERVICE_RESTART
FS-03	STORAGE_EXHAUSTION	30 % / 40 % / 96 % / 50 ms	StorageUtilization ≥ 90	STORAGE_CLEANUP
FS-04	NETWORK_LATENCY	20 % / 25 % / 15 % / 750 ms	NetworkLatency ≥ 500	NETWORK_REROUTE
FS-05	SERVICE_DOWNTIME	0 % / 0 % / 10 % / 9999 ms	NetworkLatency ≥ 500	FAILOVER

Note that FS-02, FS-04 and FS-05 share one detecting signal (NetworkLatency). When the event arrives by the alarm path, the failure type is parsed from the alarm name, so the alarm `cloudpulse-API-001-NETWORK_LATENCY` is mapped to NETWORK_LATENCY; the fast path carries the exact failure type. This is why the fast path is the authoritative route for demonstrating FS-02 and FS-05.

3.2.2 Resource state machine

A resource moves through eight states; the diagram and table show the allowed transitions.

Figure 3: Resource state machine used by the recovery engine



From	To	Trigger
HEALTHY / WARNING	FAILURE_DETECTED	Failure injected (only injectable from HEALTHY or WARNING)
FAILURE_DETECTED	RECOVERY_INITIATED	Recovery Lambda passes the idempotency gate (conditional write)
RECOVERY_INITIATED	RECOVERY_IN_PROGRESS	Strategy dispatched
RECOVERY_IN_PROGRESS	RECOVERED	Strategy succeeded; metrics reset to recovered values
RECOVERY_IN_PROGRESS	RECOVERY_FAILED	Strategy raised an error and attempts < 3
RECOVERY_FAILED	RECOVERY_INITIATED	Retry (allowed by the gate)
RECOVERY_FAILED	MANUAL_INTERVENTION_REQUIRED	Attempts exhausted (3); incident ESCALATED
RECOVERED (or any state)	HEALTHY	Manual reset / recover endpoint restores

		nominal metrics

3.2.3 Data model

Three DynamoDB tables hold all state.

Table	Key schema	Selected attributes
cloudpulse-resources-{env}	PK `resource_id`	resource_type, description, current_state, health_status, active_failure_type, four metrics, created_at, updated_at, last_heartbeat, tags
cloudpulse-incidents-{env}	PK `incident_id`; GSI `ResourceIndex` on `resource_id` (projection ALL)	failure_type, severity, status, state_at_detection / resolution, detected_at, recovery_started_at, recovered_at, resolved_at, escalated_at, recovery_action, recovery_result, notification_status, notified_transitions, retry_count, recovery_attempts, recovery_actions[], recovery_notes[], duration_seconds, error_message (the model has more than 25 fields)
cloudpulse-metrics-{env}	PK `resource_id`, SK `window_key`	Reliability snapshots per resource and time window

Incident severity is LOW, MEDIUM, HIGH or CRITICAL; incident status is OPEN → RECOVERING → RESOLVED, or ESCALATED when automatic recovery is exhausted. Each recovery attempt is stored as an embedded `RecoveryAction` (action_id, action_type, status PENDING / IN_PROGRESS / SUCCEEDED / FAILED, started_at, completed_at, outcome_message, error_detail), and the model enforces `recovery_attempts == len(recovery_actions)`.

3.2.4 Recovery handler algorithm

The handler in `lambda/recovery/handler.py` follows this outline (pseudocode):

```
handler(event):
  resource_id, failure_type = parse_event(event)        # alarm path or fast path
  if alarm name ends in SERVICE_HEALTH: return ignored   # composite signal, no strategy
  resource = read(resource_id)
  if resource.state not in {FAILURE_DETECTED, RECOVERY_FAILED}:
      return "already recovering / recovered"            # layer-1 gate
  conditional_update(state -> RECOVERY_INITIATED,
      condition = state in accepted states)              # layer-2 gate (DynamoDB)
  incident = open_or_create(resource, failure_type); notify(recovery_started)
  state -> RECOVERY_IN_PROGRESS
  strategy = dispatch_strategy(failure_type)
  try:   outcome, delta = strategy.execute(resource_id)
         state -> RECOVERED; apply delta; incident -> RESOLVED; notify(recovered)
  except: attempts += 1
         if attempts < MAX_RECOVERY_ATTEMPTS: state -> RECOVERY_FAILED  (retry possible)
         else: state -> MANUAL_INTERVENTION_REQUIRED; incident -> ESCALATED
```

3.2.5 Strategy layer and the real-AWS safety gate

`dispatch_strategy(failure_type)` returns a `RecoveryStrategy` with a recovery action type, a description and a modelled duration. `execute()` first computes the metric delta used to reconcile telemetry, then calls `_execute_real_aws_action()`, which returns `None` (simulated outcome) unless **all** of the following hold: `EXECUTION_MODE` is HYBRID or REAL_AWS, the resource ID is in `REAL_AWS_ALLOWLIST`, and the ID has a recognisable form

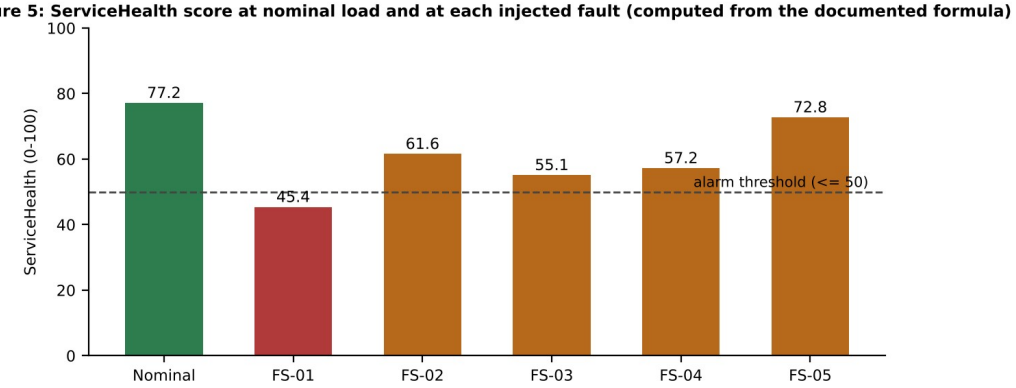
(EC2 instance ID, `asg-` prefix, `rds-`/`db-` prefix). Any AWS client error is logged and the simulated outcome is used instead.

Action	Simulated outcome (default)	Real action when allow-listed (boto3)	Duration modelled
SCALE_OUT	Increased virtual CPU allocation	`autoscaling.set_desired_capacity` (+1, HonorCooldown) on an ASG; otherwise read-only EC2 describe	3 s
SERVICE_RESTART	Health check resumed	`ec2.reboot_instances` for an EC2 instance ID (a reboot; no SSM or in-guest service restart). For any other ID only `describe_instances` runs	5 s
STORAGE_CLEANUP	Freed 30 % capacity	`ssm.send_command` (AWS-RunShellScript) running `df -h`, `du -sh /tmp /var/log`, `journalctl --disk-usage` — diagnostics only, nothing is deleted; the command ID is returned but its output is never read	4 s
NETWORK_REROUTE	Alternate path active	`elbv2.modify_listener` forwarding the first listener of `alb-cloudpulse-gw` to `tg-cloudpulse-standby` (names hard-coded); target health is not checked before or after	2 s
FAILOVER	Standby promoted	`ec2.reboot_instances` (EC2 ID) or `rds.reboot_db_instance(ForceFailover=True)`, falling back to a plain reboot if the forced failover call raises (a single-AZ instance rejects it); no post-action health check	6 s

3.2.6 Health scoring and reliability formulas

ServiceHealth is a weighted headroom score in [0, 100]: `100 × (0.30·(1−cpu/85) + 0.25·(1−mem/85) + 0.20·(1−storage/90) + 0.25·(1−min(latency/500, 1)))`, with each ratio clamped to 1. Evaluating this formula on the five fault profiles gives the values in Figure 5.

Figure 5: ServiceHealth score at nominal load and at each injected fault (computed from the documented formula)



Reliability metrics are pure functions over stored incidents and resources (`reliability_calculator.py`): incident count; recovery success and failure rate; average recovery time and MTTR; average detection time; incident frequency and MTBF over a window (with overlap clamping of incidents that straddle the window); health distribution (HEALTHY / DEGRADED / CRITICAL counts); and a per-failure-type count. Nothing is estimated: if there is no completed incident, the time-based metrics are reported as absent rather than zero.

3.2.7 REST API

The FastAPI application exposes the endpoints below through API Gateway.

Method and path	Purpose
GET /health, /health/ready	Liveness and readiness probes
GET /resources, /resources/{id}	List resources; one resource with metrics and state
GET /incidents, /incidents/{id}	List incidents (filters); incident detail with recovery actions
GET /metrics, /metrics/overview, /metrics/summary	Fleet reliability overview (summary is an alias)
GET /metrics/{resource_id}, /metrics/{resource_id}/{window_key}	Latest or specific snapshot for a resource
POST /simulate/failure (alias /simulate/inject)	Inject a scenario; returns 201 with resource, incident and emitted metrics
POST /simulate/reset/{id}, /simulate/reset, /simulate/recover	Manually return a resource to HEALTHY

Errors use a common envelope produced by registered exception handlers: 404 for an unknown resource or incident, 409 for an invalid state transition (for example injecting into a resource that is already failing), and 422 for schema violations handled by Pydantic.

3.2.8 Frontend design

The dashboard (`frontend/src/pages/Dashboard.tsx`) is organised as nine tab sections: Overview, Resource Health, Failure Simulator, Active Incidents, Incident Details, Recovery Activity, Reliability Metrics, Architecture View and System Events. A typed API client (`src/api/*.ts`) wraps Axios with retry and error normalisation; explicit loading, empty and error views are shared components. Polling is adaptive: 3 s while an incident or degraded resource is present, 10 s when the fleet is idle, and suspended entirely while the browser tab is hidden; an in-flight lock prevents overlapping polls. A five-stage lifecycle stepper (Injected, Detected, Recovering, Recovered, Notified) makes the closed loop visible during a demonstration.

3.2.9 Scenario walkthroughs (designed behaviour)

The table traces each scenario through the pipeline using the fault profiles, alarm definitions and strategy mapping in the source. It describes the designed behaviour; it is not a log of one measured run.

Scenario	Typical target	Injected change	Detecting alarm	Recovery strategy	Post-recovery
FS-01 High CPU / Capacity Pressure	VM-001	CPU 25 → 92 %; ServiceHealth 77.2 → 45.4	cloudpulse-VM-001-HIGH_CPU (CPU ≥ 85, 2 × 60 s) and cloudpulse-VM-001-SERVICE_HEALTH (ignored by handler)	SCALE_OUT: +1 desired capacity on an allow-listed ASG, otherwise virtual CPU allocation raised	CPU back to 25 %, memory 30 %
FS-02 Service Failure / Process Crash	API-001	Latency 20 → 980 ms; CPU 15 %	cloudpulse-API-001-NETWORK_LATENCY (≥ 500)	SERVICE_RESTART: `ec2.reboot_instances` on an allow-listed EC2 instance (a reboot, not an in-OS service restart; no SSM service-recovery path exists in the code), otherwise virtual restart	Latency back to 15 ms, CPU 25 %
FS-03 Storage Exhaustion	STORAGE-001	Storage 20 → 96 %	cloudpulse-STORAGE-001-	STORAGE_CLEANUP: SSM diagnostics on an allow-listed	Storage reduced by 20

			STORAGE_EXHAUSTION (≥ 90)	instance, otherwise virtual cleanup (30 % freed)	points to the recovered value
FS-04 Network Latency / Unhealthy Target	VM-001	Latency 15 → 750 ms	cloudpulse-VM-001-NETWORK_LATENCY (≥ 500)	NETWORK_REROUTE: ALB listener default action switched to the standby target group when both exist (no target-health check), otherwise virtual reroute	Latency back to 15 ms
FS-05 Database Downtime / Failover	DB-001	CPU 0 %, memory 0 %, latency 9999 ms	cloudpulse-DB-001-NETWORK_LATENCY (≥ 500)	FAILOVER: RDS reboot with `ForceFailover=True`, falling back to a plain reboot on error, for an allow-listed `rds-`/`db-` ID, EC2 reboot for an instance ID, otherwise virtual standby promotion	All four metrics reset by the failover delta

In every case the sequence of states is HEALTHY → FAILURE_DETECTED → RECOVERY_INITIATED → RECOVERY_IN_PROGRESS → RECOVERED → HEALTHY, the incident goes OPEN → RECOVERING → RESOLVED, and four SNS lifecycle messages are produced (failure detected, recovery started, recovered; recovery failed only on error).

3.2.10 Example API interaction

Failure injection accepts both camelCase (`resourceId`, `failureType`) and snake_case (`resource_id`, `failure_type`) field names, because the request model declares aliases with `populate_by_name=True`; `severity` and `parameters` are optional, and `failureType` may be the enum name or the scenario code (for example `FS-01`). The two bodies below are equivalent.

```
POST /dev/simulate/failure
{ "resourceId": "VM-001", "failureType": "HIGH_CPU" }
{ "resource_id": "VM-001", "failure_type": "HIGH_CPU" }   // same request, snake_case

201 Created
{ "message": "Simulated failure 'FS-01 High CPU Utilization' successfully injected on 'VM-001'",
  "scenario_id": "FS-01", "scenario_name": "FS-01 High CPU Utilization",
  "resource": { "resource_id": "VM-001", "current_state": "FAILURE_DETECTED", ... },
  "incident": { "incident_id": "<uuid>", "status": "OPEN", ... },
  "metrics_emitted": { "CPUUtilization": 92.0, ... } }
```

3.2.11 Notification messages

`NotificationService` builds plain-text e-mail messages with the subject patterns `[CloudPulse] FAILURE DETECTED: {resource} — {failure_type}`, `[CloudPulse] RECOVERY STARTED: …`, `[CloudPulse] RECOVERED: …` and `[CloudPulse] RECOVERY FAILED: …`. Each body carries the incident ID, correlation ID, resource, failure type, severity, status and the relevant timestamps, followed by action name, duration or error where applicable. A transition is published only once per incident (tracked in `notified_transitions`), and an SNS failure is logged without failing recovery.

3.2.12 Reliability metric definitions

All reliability figures are computed from stored incident and resource records using these definitions.

Metric	Definition used in the code	Absent when

Incident count	Number of incidents in the window	Never (0 allowed)
Recovery success rate	RESOLVED incidents / incidents that reached a terminal outcome (RESOLVED or ESCALATED)	No terminal incidents
Recovery failure rate	ESCALATED incidents / incidents that reached a terminal outcome	No terminal incidents
Average recovery time	Mean over RESOLVED incidents of the sum of succeeded action durations; falls back to recovered_at − recovery_started_at, then duration_seconds	No resolved incidents
MTTR	Mean over RESOLVED incidents of duration_seconds, or resolved_at − detected_at	No resolved incidents
Average detection time	Mean of max(0, detected_at − created_at)	No incidents
Availability	(1 − downtime / window) × 100, counting only the part of each outage that overlaps the window; unresolved outages run to the window end	No closed incident window
Incident frequency	Incidents per hour and per day over the window (window span at least 60 s)	Never
MTBF	Window span in seconds divided by incident count	No incidents
Health distribution	Counts of resources that are HEALTHY, DEGRADED or CRITICAL	Never

The MTBF row summarises the documented intent (window span over count); the exact rounding and the overlap clamping for incidents that straddle a window boundary are implemented in `_overlap_seconds` and `calculate_incident_frequency`.

3.2.13 EventBridge rules (as deployed in the template)

The Recovery Lambda subscribes to two EventBridge rules, defined as follows in the template.

```
# Rule 1 - alarm path
{ "source": ["aws.cloudwatch"],
  "detail-type": ["CloudWatch Alarm State Change"],
  "detail": { "alarmName": [{"prefix": "cloudpulse-"}],
              "state": { "value": ["ALARM"] } } }

# Rule 2 - direct fast path
{ "source": ["cloudpulse.simulator"], "detail-type": ["FailureInjected"] }

# fast-path detail consumed by the handler
{ "resourceId": "VM-001", "failureType": "HIGH_CPU", "correlationId"/"incidentId": "<uuid>" }
```

3.2.14 Data access patterns

The repositories access the tables as shown below.

Operation	Table / index	DynamoDB call	Used by
Read one resource	resources (PK resource_id)	GetItem (strongly consistent)	API, recovery pre-read
List resources	resources	Scan (four items)	Overview, health sections
Guarded state change	resources	UpdateItem with ConditionExpression on current_state	Recovery gate, simulation, reset
Create / update incident	incidents (PK incident_id)	PutItem / UpdateItem	Simulation service, recovery
Incidents of one resource	incidents (GSI ResourceIndex is defined)	Scan with filter; the GSI is not queried yet (audit A-04)	Recovery incident lookup, resource views
List incidents with filters	incidents	Scan with limit ≤ 200	Incident list endpoint

Metric snapshots	metrics (PK resource_id, SK window_key)	Query / GetItem / PutItem	Reliability endpoints

The table is derived from the key schemas and the IAM actions granted to each role (GetItem, Scan, Query, PutItem, UpdateItem). With four resources and a student-scale incident volume, Scan is acceptable. The ResourceIndex GSI is defined in the template but the repository still scans (a TODO recorded as audit finding A-04), so per-resource incident lookups would need a code change to benefit from it at larger scale.

3.2.15 CloudWatch metrics and dimensions

Five custom metrics are published for every resource; four of them drive alarms.

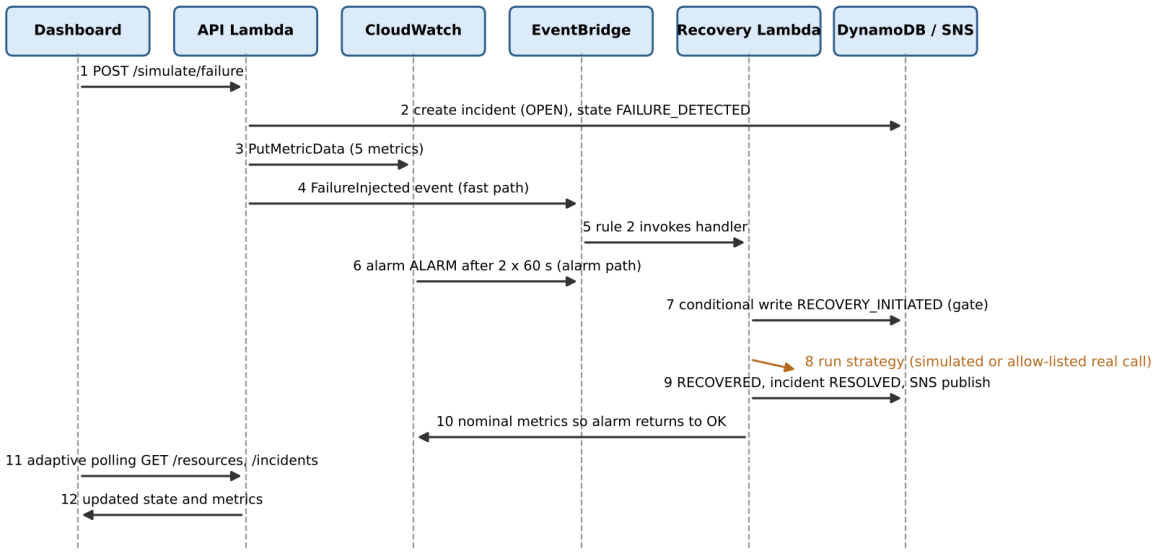
Metric (namespace CloudPulse)	Unit	Alarm	Notes
CPUUtilization	Percent	≥ 85 on VM-001, API-001, DB-001	Average over 60 s
MemoryUtilization	Percent	None (input to ServiceHealth)	Published for every resource
StorageUtilization	Percent	≥ 90 on STORAGE-001	Average over 60 s
NetworkLatency	Milliseconds	≥ 500 on all four resources	Shared by FS-02, FS-04, FS-05
ServiceHealth	None (0–100)	≤ 50 on all four resources	Composite; recovery handler ignores its alarms

Each datum carries the dimensions `ResourceId` and `ResourceType`. Alarms use the same dimensions, so a metric published for one resource cannot trigger another resource's alarm.

3.2.16 Interaction sequence

The diagram shows the order of calls for one injected failure.

Figure 6: Sequence of one injected failure (fast path and alarm path)



Steps 3–6 show that the same fault can reach the handler by two routes; the idempotency gate in step 7 guarantees that only the first arrival performs the recovery. The metric reset

in step 10 lets the alarm return to OK so that a later injection produces a new ALARM transition.

3.2.17 Dashboard sections

Section	Content shown
Overview	Counts of healthy, warning, failed and recovering resources; active incidents; recovery success rate; MTTR; alert banner; five newest incidents
Resource Health	Filter by type; grid or table; cards with CPU, memory, storage and latency gauges, state and health badges, heartbeat age; Inject Fault and Reset buttons
Failure Simulator	Choose target resource, scenario FS-01 to FS-05 and severity; trigger and reset; blueprint card explaining metric, threshold and strategy; output receipt with incident ID
Active Incidents	Filter by status, severity, failure type or text; table with detection time, duration and recovery action; rows open the detail view
Incident Details	Lifecycle stepper (detected, recovery initiated, action executed, resolved or escalated); incident fields; recovery-action audit log; notification panel
Recovery Activity	Action totals, successes, failures, average duration; distribution by strategy; chronological execution stream
Reliability Metrics	Availability, MTTR, MTBF and self-healing success rate; daily, weekly or cumulative window; per-resource breakdown; scenario distribution
Architecture View	Interactive closed-loop diagram with state-dependent highlighting and a specification drawer per node
System Events	Chronological stream of detections, recovery starts, resolutions and notifications with category and resource filters

One cosmetic inconsistency: the Architecture View labels the metric namespace `CloudPulse/SimulatedFleet`, whereas the template default (and the IAM condition) is `CloudPulse`.

4. Implementation : 80%+ (final implementation)

This section reports what has been implemented in the repository, how it was verified, its cost and security posture, and its known limits.

4.1 Implementation status

The template requires at least 80 % of the planned system to be implemented at Review II. All planned functional components of the design in Section 3 are implemented and present in the repository at the time of writing; the items that are deliberately not implemented are listed in 4.9 so that the status is not overstated. Verification figures quoted below are those reported by the repository (README and `docs/REVIEW_2`); the frontend count was re-counted from the test sources for this document (49), the backend count was not re-executed in this environment.

Component	Status	Where / how verified
SAM template: 3 Lambdas, layer, HTTP API, 3 DynamoDB tables, 12 alarms, 2 EventBridge rules, SNS, SQS DLQ, S3, 3 IAM roles, log groups	Implemented	`infrastructure/template.yaml` (961 lines); `sam validate --lint` reported passing
FastAPI backend (health, resources, incidents, metrics, simulate)	Implemented	`backend/app` (routers, services, repositories, models)
Failure simulation engine, 5 scenarios	Implemented	`services/simulation_service.py`, `models/resource.py`
Simulator Lambda (heartbeat, batched metrics)	Implemented	`lambda/simulator/handler.py`; 24 test functions
Recovery Lambda: parse, gate, strategies, audit,	Implemented	`lambda/recovery`; 21 + 13 + 9 test

retry, escalate		functions
Real-AWS hybrid remediation behind allow-list (v2)	Implemented	`lambda/recovery/strategies.py`; `scripts/manage_real_aws.py`; live behaviour documented in `docs/CLOUDPULSE_V2_REAL_AWS.md` (not re-verified here)
Incident lifecycle and SNS notifications with de-duplication	Implemented	`services/notification_service.py`
Reliability calculator (8 metric families)	Implemented	`services/reliability_calculator.py`; 10 + 5 test functions
React dashboard (9 sections), typed API client, adaptive polling	Implemented	`frontend/src`; 49 frontend tests
CI (backend pytest, frontend vitest + build)	Implemented	`.github/workflows/ci.yml`
Authentication, WAF, CloudFront, KMS CMKs, PITR	Not implemented (by design)	Residual risks, Section 4.8

4.2 Repository layout

The repository is organised as follows.

```
cloudpulse/
  backend/app/        FastAPI app: routers, services, repositories, models, aws clients, logging
  lambda/api|simulator|recovery/   Lambda entry points (+ recovery/strategies.py)
  frontend/src/       React 18 + TypeScript dashboard, api client, components, tests
  infrastructure/     template.yaml (SAM), samconfig.toml, IAM policy notes
  scripts/            deploy.sh, deploy_frontend.sh, seed_data.py, reset_resources.py,
                      manage_real_aws.py, validate.sh, destroy.sh
  tests/              api, aws_integration, e2e, edge_cases, eventbridge, integration,
                      notification, recovery, simulation, unit (+ fixtures)
  backend/tests/unit/ unit tests for models, services, handlers
  docs/               architecture, API, DB, security, cost, test strategy, ADRs, review docs
  .github/workflows/ci.yml
```

The Python source in `backend/app` and `lambda` is roughly 6,500 lines; the TypeScript/React source and its tests are roughly 5,900 lines (line counts from `wc -l`, indicative only).

4.3 Backend and API implementation

Main implementation points of the backend:

- **Application factory and Lambda adapter.** `backend/app/main.py` builds the FastAPI app with CORS middleware (origins from settings, credentials disabled) and registered exception handlers; `lambda/api/handler.py` wraps it with Mangum (`lifespan="off"`) and strips the API Gateway stage prefix from the path – the fix for the 404 on `/dev/simulate/failure` found during deployment (commit 8d28d41).
- **Layered design.** Routers handle HTTP only; services (`SimulationService`, `IncidentService`, `ResourceService`, `MetricService`, `MonitoringService`, `NotificationService`) hold business rules; repositories wrap DynamoDB; Pydantic models validate every boundary. This keeps the Lambda handlers thin and the logic unit-testable without AWS.

- **Validation.** Resource IDs, incident IDs, pagination limits and failure-type enums are validated before any AWS call; metric overrides supplied to the simulator are bounded (0–100 for utilisation, 0–99 999 ms for latency).
- **Non-fatal telemetry.** CloudWatch and EventBridge calls in `MonitoringService` log a warning on failure instead of raising, so a monitoring outage cannot block the API.
- **Structured logging.** `logging_config.py` emits single-line JSON with correlation ID, incident ID and trace stage, so one incident can be followed across the API, EventBridge and Recovery Lambda log groups.

4.4 Simulator and monitoring implementation

The Simulator Lambda runs every two minutes. For each resource it applies a small random-walk jitter to the nominal metrics, computes `ServiceHealth` and publishes the five metrics (CPUUtilization, MemoryUtilization, StorageUtilization, NetworkLatency, ServiceHealth) to namespace `CloudPulse` with dimensions `ResourceId` and `ResourceType`, in **one batched `put_metric_data` call per resource** (5 metrics per call). This reduces CloudWatch API calls fivefold compared with one call per metric. Resources that are in a recovery state are skipped by the heartbeat, and a resource already in FAILURE_DETECTED is never moved back to HEALTHY by it. This heartbeat is the telemetry source in SIMULATION mode, where it publishes synthetic metrics for the four virtual resources. In HYBRID / REAL_AWS deployments the real fleet reports its own native EC2, RDS and ALB CloudWatch metrics, so the simulator is optional for those resources (and random-walk jitter must not be mistaken for a measurement of the live instance).

4.5 Self-healing engine implementation

The Recovery Lambda implements the algorithm in Section 3.2.4. Key implementation points verified in the source:

- **Event parsing** supports both event shapes. For alarm events the failure type is recovered from the alarm-name suffix, scanning from the right so multi-hyphen IDs such as `STORAGE-001` parse correctly. A parse failure is **re-raised** (not returned as HTTP 400) so that Lambda retry and the SQS DLQ engage; returning normally would have silently lost the event (audit finding A-06).
- **SERVICE_HEALTH alarms** are acknowledged and ignored; they signal fleet degradation but do not select a strategy. The EventBridge pattern matches the prefix `cloudpulse-` and the filtering is done in the handler.
- **Idempotency.** Layer 1 reads the resource and exits unless it is in FAILURE_DETECTED or RECOVERY_FAILED. Layer 2 performs `update_state` with `condition_states`, so two concurrent events cannot both win; the loser exits with “Already recovering”. A DynamoDB error during this phase is re-raised for retry.
- **Incident reuse.** If an OPEN or RECOVERING incident exists for the resource (created at injection) it is reused; otherwise a new incident is created in RECOVERING state, which covers events that originate outside the API.

- **Bounded retries.** `MAX_RECOVERY_ATTEMPTS = 3`. Each attempt is appended to `recovery_actions`; after the third failure the resource moves to MANUAL_INTERVENTION_REQUIRED and the incident to ESCALATED.
- **Recovery metrics.** On success, nominal metrics are pushed back to CloudWatch (non-fatal) so that alarms can return to OK.

Real-AWS remediation gate (excerpt)

The two checks that guard every real AWS call are implemented at the top of `_execute_real_aws_action` in `lambda/recovery/strategies.py`:

```
_configured_mode = os.environ.get("EXECUTION_MODE", "SIMULATION").upper()
EXECUTION_MODE = _configured_mode if _configured_mode in {"SIMULATION","HYBRID","REAL_AWS"} \
                 else "SIMULATION"                          # unknown value falls back safely
REAL_AWS_ALLOWLIST = {i.strip().upper() for i in
                      os.environ.get("REAL_AWS_ALLOWLIST", "").split(",") if i.strip()}

def _execute_real_aws_action(self, resource_id):
    if EXECUTION_MODE == "SIMULATION":            return None   # gate 1
    if resource_id.strip().upper() not in REAL_AWS_ALLOWLIST: return None   # gate 2
    ...                                           # boto3 call; ClientError -> simulated outcome
```

4.6 Frontend implementation

The dashboard is a Vite-built React 18 single-page application written in TypeScript with Tailwind CSS. The nine sections (Overview, Resource Health, Failure Simulator, Active Incidents, Incident Detail, Recovery Activity, Reliability Metrics, Architecture View, System Events) are separate components under `components/sections`, composed by `pages/Dashboard.tsx`, which owns polling and state. Shared components provide cards, gauges, badges, charts and the loading / empty / error views. The Axios client has a 10 s timeout and retries only idempotent GET/HEAD requests (300 ms, 600 ms back-off) on 502/503/504 or network errors; POST requests are never retried, which prevents duplicate injections. The header shows API connectivity, the current refresh rate and a “v2 • Real AWS & Hybrid” label; v2 also adjusts the resource-health, simulator and incident-detail sections to display real-resource information.

4.7 Testing and verification

`docs/TEST_STRATEGY.md` defines nine testing layers plus an edge-case suite; the repository contains a matching directory for each. The table counts test functions per area from the sources (parametrised cases expand the backend total).

Layer / area	Location	Test functions	What is checked
Unit	`backend/tests/unit`, `tests/unit`	278	Models and validation (116 in `test_models.py`), config, repositories, services, simulator and recovery handlers, strategies, reliability calculator, event parsing
API	`tests/api`	5	Endpoints, status codes, validation
Integration	`tests/integration`	4	Service + repository wiring (moto)
AWS integration	`tests/aws_integration`	5	CloudWatch, EventBridge, SNS, DynamoDB clients
Simulation	`tests/simulation`	3	Five-scenario fault profiles

EventBridge flow	`tests/eventbridge`	5	Event patterns and both routes
Recovery	`tests/recovery`	3	Strategy dispatch and state transitions
Notification	`tests/notification`	4	SNS lifecycle messages and de-duplication
End-to-end	`tests/e2e`	1	Closed loop from injection to RESOLVED
Edge cases	`tests/edge_cases`	10	Strategy failure, retry exhaustion, duplicate events, invalid input, CloudWatch/SNS outage, conditional-write conflict
Frontend (Vitest)	`frontend/src/tests`	49	Components, sections, navigation, API client, dashboard

Executable suite	Location	Result (owner-run output, `docs/VERIFICATION_APPENDIX.md`)
Root integration and system	`tests/`	95 passed
Backend domain and services	`backend/tests/`	273 passed, 5 skipped
Frontend Vitest	`frontend/src/tests/`	49 passed (17 test files)
Total	—	**417 passed of 422 collected (5 skipped)**
TypeScript `tsc --noEmit` / `vite build`	`frontend/`	0 errors
`mypy app/`	`backend/` (30 source files)	0 errors
`sam validate --lint`	`infrastructure/template.yaml`	valid

The three suites above are the executable totals; the table before them groups the same tests by conceptual layer, and its counts are static function counts from the source. The pasted pytest output (368 passed, 5 skipped) and Vitest output (49 passed in 17 files) were supplied by the project owner and were not re-run for this document. The five skipped tests are described in the repository as conditional on live AWS credentials or an optional simulation flag.

Resilience cases covered by the edge-case suite

Failure handling is verified by the edge-case suite (`tests/edge_cases`):

Case	Expected behaviour
Recovery strategy raises an error	Attempt recorded as FAILED; resource RECOVERY_FAILED; retry allowed while attempts < 3
Retries exhausted	MANUAL_INTERVENTION_REQUIRED; incident ESCALATED; failure notification sent
Duplicate EventBridge delivery	Second invocation exits with “Already recovering”; one recovery action only
Unknown resource	API returns 404; handler error is re-raised for retry/DLQ
Invalid failure type	Schema validation error (422)
CloudWatch outage while injecting	Warning logged; injection and incident creation still succeed
SNS failure	Warning logged; recovery result still persisted
Concurrent conditional-write conflict	One writer wins; the other is a no-op and exits

4.8 Security implementation

Security controls are summarised against the STRIDE categories used in `docs/SECURITY.md`.

STRIDE	Mitigation implemented	Residual note
Spoofing	Recovery Lambda accepts only the two expected sources and parses alarm names strictly	Anyone with `events:PutEvents` in the account could still emit a matching event
Tampering	Separate IAM roles; conditional expressions on state	IAM cannot restrict which attribute

	updates; strongly consistent reads	values a role writes
Repudiation	Incident document with per-attempt audit records, timestamps and notification log	—
Information disclosure	Structured logs without credentials; access logs omit headers and bodies; `.env` ignored by git	—
Denial of service	Pagination limit ≤ 200; state-machine validation blocks repeated injection; bounded Lambda timeouts; CORS preflight cache	No throttling/WAF; the API is unauthenticated
Elevation of privilege	No `iam:*`; PutMetricData restricted to namespace `CloudPulse` by condition key	Recovery role carries broad real-AWS actions on all resources (v2); the limit is the code-level allow-list

Other trade-offs accepted for a demonstration and documented in `docs/SECURITY.md`: open API routes with no authorizer, no WAF, default AWS-owned encryption keys, point-in-time recovery disabled, and S3 static website hosting (the bucket is private by default and becomes public only with `EnablePublicFrontendBucket=true`). The CORS origin defaults to `http://localhost:5173`; a deployment that serves the dashboard from S3 must pass the website origin as `CorsAllowedOrigins`.

4.9 Results, cost and known limitations

This section gives the cost estimate, the evidence recorded by the project and the limits of the delivered system.

Cost

The repository gives three different monthly cost figures (below US$1 in `COST_MANAGEMENT.md`, US$25–35 and US$35–45 in `CLOUDPULSE_V2_REAL_AWS.md`). They are not consistent with each other or with the code, so they were recomputed here from the repository's own unit prices. These are estimates, not billing data.

Item	Basis	Estimate / month
Serverless control plane	20 custom metrics (4 resources × 5 metrics; the older doc counts 16), 10 free then US$0.30 each; 12 alarms, 10 free then US$0.10 each; Lambda, DynamoDB, EventBridge, SNS, SQS, S3 within or near Free Tier	≈ US$3.2 (3.00 + 0.20)
EC2 t3.micro	$0.0104 / h × ~720 h (v2 doc: ~$7.50)	≈ US$7.5
Auto Scaling Group (1–3 instances)	Same instance price, 1–2 extra instances	≈ US$7.5–15
Application Load Balancer	Fixed hourly charge plus LCU-hours; v2 doc: ~$16.20	≈ US$16.2
RDS PostgreSQL 16 db.t4g.micro + 20 GB gp3	Instance-hours plus storage; v2 doc: ~$14.50	≈ US$14.5
SSM and S3 website endpoint	Run Command and managed-instance registration are free at this scale; S3 traffic is negligible	≈ US$0
Total with the v2 fleet running continuously	Before any Free Tier credit	**≈ US$49–56**

At that rate a US$100 credit lasts roughly two months, not longer, so `python scripts/manage_real_aws.py teardown-fleet` should be run when the demonstration ends (it deletes the ALB, ASG, RDS instance and tagged EC2 instances; RDS is deleted with `SkipFinalSnapshot=True`).

Behaviour verified by the project

Evidence recorded by the project:

- All five scenarios were exercised end to end in tests (injection → incident OPEN → RECOVERING → RESOLVED, resource back to HEALTHY, notification recorded).
- v2 documentation records one live evidence item per scenario: ASG desired capacity 1 → 2 with a new instance; EC2 reboot; an SSM command with status Success; ALB listener switched to the standby target group; RDS entering `rebooting`. These were taken from `docs/CLOUDPULSE_V2_REAL_AWS.md` and were not re-verified in this environment.

Limitations (stated plainly)

The following limits apply to the delivered system.

- **Recovery time is time-to-dispatch, not time-to-healthy.** The 2–6 s strategy durations are SIMULATION defaults. In the live deployment `duration_seconds` is `detected_at` to `resolved_at` from stored timestamps. Six real remediations recorded on 24 October 2026 (Section 4.15) range from 3.23 s to 6.80 s (mean 4.40 s, median 3.61 s; owner-reported, not re-queried here). The README's 4.8 s figure does not match that mean and should be changed to 4.40 s or its source stated. The clock stops when the AWS call returns and the incident is closed; no health check runs afterwards, so EC2 boot, RDS availability and target-group convergence are excluded. The viva-note claim that recovery is faster than commercial tools is not a controlled comparison.
- **Real actions report more than they verify.** In the code, FS-02 only reboots an EC2 instance (there is no SSM service-restart path); FS-03 sends read-only SSM diagnostics (`df -h`, `du -sh /tmp /var/log`, `journalctl --disk-usage`) and never reads their output or deletes anything; the RDS path tries `ForceFailover=True` and silently falls back to a plain reboot on error, which is what the captured incident message ("rebooted") indicates; and some fallback messages ("service restarted", "failover active") are returned when no real change was made. The README's "disk log reclamation", "Multi-AZ failover" and "zero dropped requests" statements are not supported by the code or the captured evidence.
- **Real fleet is not defined in the SAM template.** The EC2/ASG/ALB/RDS resources are created outside it (the CLI provisions only a demo EC2 instance and registers/inspects/tears down the rest; no script in the repository creates the ASG, ALB or RDS instance), and the 12 CloudWatch alarms cover only the four virtual resources. Real-fleet recovery is therefore triggered through the fast path (`POST /simulate/failure`), not by genuine CloudWatch alarms on those resources.
- **ServiceHealth weakness** for outage cases (Section 3.2.6): a total outage scores 72.8, so the composite alarm does not detect it.
- **Repository text drift.** Several documents still describe the system as 100 % simulated (for example the Recovery Lambda module docstring and the "cardinal safety

boundary" in the README) while v2 adds opt-in real actions; these statements are true for the default SIMULATION mode only.

- **Scale.** Four resources and single-region operation; production-scale tracing, very-large-fleet indexing and true wall-clock recovery measurement are out of scope.

4.10 Build, deploy and demonstration procedure

Commands to build, test and deploy the project, followed by the demonstration flow:

```
# backend and tests
pip install -r backend/requirements-dev.txt
PYTHONPATH=backend:. python -m pytest tests/ backend/tests/ -q
# frontend
cd frontend && npm ci && npm test -- --run && npm run build
# infrastructure (SIMULATION is the default execution mode)
sam validate --lint -t infrastructure/template.yaml
./scripts/deploy.sh -e dev -m <email>   # optional: -x HYBRID -a <id,...>  (default mode SIMULATION)
python scripts/seed_data.py --table cloudpulse-resources-dev --region ap-south-1
./scripts/deploy_frontend.sh dev
# optional v2 fleet
python scripts/manage_real_aws.py status | discover | teardown-fleet
```

Demonstration flow: open the dashboard; show the four resources HEALTHY; inject FS-01 on VM-001; watch the five-stage stepper and the 3 s polling; open the incident detail to show the recovery action list and timestamps; check the reliability metrics page and the SNS e-mail; finally show the CloudWatch alarms and Lambda logs carrying the same correlation ID.

4.11 Risk register

Project risks and their mitigations:

Risk	Likelihood	Impact	Mitigation in the project
Real remediation touches the wrong resource	Low	High	Execution mode must be non-SIMULATION and resource ID must be allow-listed; unknown IDs fall back to simulation; teardown CLI tags-based
Unexpected AWS charges from the v2 fleet	Medium	Medium	Cost estimate in Section 4.9; fleet is optional; `teardown-fleet` command
Duplicate recovery from duplicate events	Medium	Medium	Two-layer idempotency; notification de-duplication
Lost event when parsing fails	Low	High	Exception re-raised so Lambda retry and DLQ engage
Unauthenticated API abused	Medium (if public)	Medium	Input validation, state-machine guards, localhost CORS default; authorizer listed as future work
Overbroad IAM on Recovery role	Certain (as built)	Medium	Documented; recommended to scope to ARNs before any non-demo use
Documentation drift	Medium	Low	Counts and claims re-checked against code for this document
Semantic overclaim: calling an EC2 reboot a "service restart", SSM diagnostics "cleanup", an ALB listener edit "network recovery" or a plain RDS reboot a "failover"	Medium (present in code messages and README)	Medium	Record the exact AWS action name in incident history; require a post-action health predicate (process/endpoint probe for FS-02, free-space and write test for FS-03, target health for FS-04, read/write transaction for FS-05) before RECOVERED. Not implemented yet

4.12 Future work

Planned improvements, in priority order:

- Scope the Recovery IAM policy to the allow-listed ARNs and attach it only when a non-simulation mode is selected.
- Define the real fleet (EC2, ASG, ALB, RDS) in the SAM template or a companion template and add CloudWatch alarms for the real resources so that real detection, not only the fast path, drives real remediation.
- Measure true wall-clock recovery (alarm time to healthy status checked from the resource) instead of using modelled durations.
- Add an API authorizer (Cognito or JWT), AWS WAF and CloudFront in front of the S3 origin.
- Revise ServiceHealth so that a total outage scores low (for example by treating zero heartbeat or very high latency as a hard zero).
- Add distributed tracing (AWS X-Ray) and a larger synthetic fleet to test indexing and scale.

4.13 Deployment parameters

The SAM template exposes these parameters; `scripts/deploy.sh` maps its options to them.

Parameter	Default	Meaning
Environment	dev	Suffix for resource names (dev, test, prod)
NotificationEmail	(empty)	If set, creates an SNS e-mail subscription
CorsAllowedOrigins	http://localhost:5173	Allowed browser origin for the API
CloudWatchNamespace	CloudPulse	Namespace of custom metrics; also the IAM condition value
LogLevel	INFO	Application log verbosity
ExecutionMode	SIMULATION	SIMULATION, HYBRID or REAL_AWS
RealAwsAllowlist	(empty)	Resource IDs that may receive real AWS calls; empty means none
EnablePublicFrontendBucket	false	Make the S3 bucket a public static website (demo only)

4.14 Audit and hardening history

`ARCHITECTURE_AUDIT.md` lists 20 findings from the architecture review; `FINAL_TECHNICAL_AUDIT.md` records the fixes. Counting the status column: 9 fixed, 7 accepted and 4 documented as known limitations. The table below lists those that affect behaviour or security.

ID	Area	Finding	Severity	Status
A-02	API Gateway	CORS default * allowed any origin	High	Fixed (localhost default)
A-03	FastAPI	PUT and DELETE allowed by CORS but unused	Medium	Fixed
A-06	Recovery Lambda	Parse error returned HTTP 400, treated as success, bypassing the DLQ	High	Fixed (re-raise)
A-10	DynamoDB	Incident update used unconditional put (race window)	High	Fixed (ConditionExpression)
A-13	CloudWatch	FS-02, FS-04 and FS-05 share one alarm, so the autonomous path cannot tell them apart	High	Documented, not fixed
A-14	EventBridge	SERVICE_HEALTH alarms matched the rule and were silently discarded	Critical	Fixed (handler filter)

A-16	Testing	No tests for event-parsing edge cases	Medium	Fixed
A-18	IAM	API role had Scan on the incidents table	Low	Fixed
A-04	DynamoDB	Incident list uses Scan; GSI defined but unused	Medium	Accepted
A-15	DynamoDB	Metrics table has no TTL	Medium	Documented
A-19	Notifications	RECOVERY STARTED can be followed by RECOVERY FAILED for the same attempt	Low	Accepted

4.15 Live deployment

The project owner reports that the stack is deployed and working in ap-south-1. All identifiers below are taken from the repository README and the owner's pasted command output; they were not independently opened by the document author. The template defaults are restrictive (private bucket, localhost CORS), so the live site presumably used `EnablePublicFrontendBucket=true` and `CorsAllowedOrigins` set to the website origin; that is an inference.

Component	Reported identifier
Frontend	`http://cloudpulse-frontend-583594224264-dev.s3-website.ap-south-1.amazonaws.com` (HTTP only, ADR-004)
API	`https://s09bowmk08.execute-api.ap-south-1.amazonaws.com/dev` (HTTP API `cloudpulse-api-dev`)
EC2 / ASG	`i-083e8c6b1fc9ca0b7` (t3.micro, AL2023, ap-south-1a); `asg-cloudpulse-fleet` (min 1, max 3, desired 2)
ALB / RDS	`alb-cloudpulse-gw` (`tg-primary`, `tg-standby`); `rds-cloudpulse-db` (PostgreSQL 16, db.t4g.micro, port 5432)
Messaging	SNS `cloudpulse-alerts`; SQS DLQ `cloudpulse-recovery-dlq`; SSM node via `cloudpulse-ec2-ssm-profile`
Recovery function / role	`cloudpulse-recovery-dev` / `cloudpulse-recovery-role-dev`

The EC2 public IP address and the ALB DNS name are intentionally omitted from this document because they are live-addressable; they can be supplied in a separate, redactable appendix if the evaluator needs them.

Measured remediations (24 October 2026)

Six real remediations were recorded in the incidents table. Each duration is `detected_at` to `resolved_at`, the time until the call was dispatched and the incident closed. The records were supplied by the project owner; the mean and median below were recomputed from the six values.

Scenario	Real action	Duration	Evidence reported
FS-01 High CPU	ASG desired capacity 1 → 2	5.87 s	Two instances listed by `describe-auto-scaling-groups`
FS-02 Service failure	`ec2:RebootInstances` (first run)	6.80 s	CloudTrail event below
FS-02 Service failure	`ec2:RebootInstances` (second run)	3.52 s	SNS RECOVERED message
FS-03 Storage exhaustion	SSM `AWS-RunShellScript` diagnostics	3.29 s	Command status Success; `df -h` output captured
FS-04 Network latency	ALB listener → `tg-cloudpulse-standby`	3.23 s	Listener default action confirmed by `describe-listeners`; restored afterwards with `scripts/restore_demo_alb.sh`
FS-05 Database downtime	`reboot_db_instance` (ForceFailover rejected, plain reboot)	3.69 s	Instance status `rebooting`
Summary (n = 6)	—	**3.23–6.80 s**	**Mean 4.40 s, median 3.61 s**

These are time-to-dispatch values. They do not include instance boot, database availability or target-group health, which can take much longer, and six samples from one day are not a statistical benchmark.

Real-action evidence

Two records were supplied by the project owner (also in `docs/VERIFICATION_APPENDIX.md`). They show that real calls happened; they do not show that the resources were healthy afterwards.

Evidence item	EC2 reboot (CloudTrail)	RDS recovery (DynamoDB incident)
boto3 call / IAM action	`ec2.reboot_instances` / `ec2:RebootInstances`	`rds.reboot_db_instance` / `rds:RebootDBInstance`
Identifier	EventID and requestID `c7a91e34-331e-4454-ad10-2f13b2ef9ce9` / `1fc24ffe-75a3-4eac-9e6d-32e9366efd60`	Incident `d5eb1604-e3d3-4aea-b748-593c4a496b4f`
Target	`i-083e8c6b1fc9ca0b7`, ap-south-1	`RDS-CLOUDPULSE-DB` (stored upper-case)
Actor	Role `cloudpulse-recovery-role-dev`, session `cloudpulse-recovery-dev`, Lambda user agent (Python 3.12)	Recovery Lambda, action FAILOVER
Time (UTC)	2026-10-24 08:18:33Z	detected 08:20:01.69Z, resolved 08:20:05.39Z (3.69 s)
Outcome text	API `_return: true`	"Successfully rebooted live RDS database" (plain-reboot path)
Mode / allow-list at run time	Not supplied	Not supplied
Pre/post health, health predicate	Not supplied; no predicate exists in code	Not supplied; no predicate exists in code
Rollback / teardown	A reboot is self-reversing; fleet removal via `teardown-fleet`	Same

The CloudTrail event is not linked here to a specific incident record, and the two records are from different actions. The ASG scale-out, SSM diagnostics and ALB listener actions are implemented behind the gate but are not independently verified in this document.

4.16 Source code excerpts

Two excerpts from the repository: the guarded state change and the alarm-name parsing.

Conditional state change (backend/app/repositories/resource_repository.py)

`update_state` builds a DynamoDB condition from the set of acceptable current states, which is what makes the second layer of the idempotency guard atomic.

```
allowed_states = set(condition_states or ())
if condition_state is not None:
    allowed_states.add(condition_state)
if allowed_states:
    placeholders = []
    for index, state in enumerate(sorted(allowed_states, key=lambda s: s.value)):
        placeholder = f":cs{index}"
        placeholders.append(placeholder)
        update_kwargs["ExpressionAttributeValues"][placeholder] = state.value
    update_kwargs["ConditionExpression"] = f"current_state IN ({', '.join(placeholders)})"
```

Alarm-name parsing (lambda/recovery/handler.py)

The failure type is recovered from the alarm name by scanning suffixes from the right, so resource IDs that contain hyphens still resolve. Anything that cannot be parsed raises, which sends the event to retry and the dead-letter queue.

```
alarm_name: str = detail.get("alarmName", "")
parts = alarm_name.split("-")
for i in range(len(parts) - 1, 1, -1):
    failure_candidate = "_".join(parts[i:]).upper()
```

```
    try:
        failure_type = FailureType(failure_candidate)
        resource_id = "-".join(parts[1:i]).upper()
        return resource_id, failure_type
    except ValueError:
        continue
raise ValueError(f"Cannot parse event: source='{source}', detail={detail}")
```

Additional Related Information

Further points that complete the submission follow.

Innovation. CloudPulse pairs a safe educational digital twin with an opt-in bridge to real EC2, Auto Scaling, load-balancer and RDS actions, guarded by an execution mode and a per-resource allow-list.

Safety boundary. In the default SIMULATION mode failures and recoveries are logical (DynamoDB state, incident records, custom metrics). Real compute or database actions occur only with `ExecutionMode` HYBRID or REAL_AWS **and** the resource listed in `RealAwsAllowlist`.

Repository: https://github.com/Chetan0246/cloudpulse (MIT licence, 32 commits at the time of writing). The deployed system is an academic MVP; authentication, production-scale tracing and verified time-to-healthy measurement are outside the current scope.

References

[1] J. Scheuner and P. Leitner, “Function-as-a-Service performance evaluation: A multivocal literature review,” J. Syst. Softw., vol. 170, 2020.
[2] P. Leitner, E. Wittern, J. Spillner and W. Hummer, “A mixed-method empirical study of Function-as-a-Service software development in industrial practice,” J. Syst. Softw., vol. 149, pp. 340–359, 2019.
[3] V. Yussupov et al., “FaaSten your decisions: A classification framework and technology review of Function-as-a-Service platforms,” J. Syst. Softw., vol. 175, 2021.
[4] A. Mampage, S. Karunasekera and R. Buyya, “A holistic view on resource management in serverless computing environments: Taxonomy and future directions,” ACM Comput. Surv., vol. 54, no. 11s, 2022.
[5] G. A. S. Cassel et al., “Serverless computing for Internet of Things: A systematic literature review,” Future Gener. Comput. Syst., vol. 128, 2022.
[6] M. Enes, R. R. Expósito and J. Touriño, “Real-time resource scaling platform for Big Data workloads on serverless environments,” Future Gener. Comput. Syst., 2020.
[7] M. Malawski, A. Gajek, A. Zima, B. Balis and K. Figiela, “Serverless execution of scientific workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions,” Future Gener. Comput. Syst., 2020.
[8] S. Ristov, S. Pedratscher and T. Fahringer, “AFCL: An abstract function choreography language for serverless workflow specification,” Future Gener. Comput. Syst., vol. 114, pp. 368–382, 2021.
[9] A. Brogi, J. Carrasco, F. Durán, E. Pimentel and J. Soldani, “Self-healing trans-cloud applications,” Computing, vol. 104, no. 4, pp. 809–833, 2022.
[10] D. Cotroneo, L. De Simone, P. Liguori and R. Natella, “Enhancing the analysis of software failures in cloud computing systems with deep learning,” J. Syst. Softw., 2021, doi:10.1016/j.jss.2021.111043.

For Evaluation only

Status ：ACCEPTED / REJECTED

Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956)	**School of Computer Science Engineering and Information Systems** **B.Tech Information Technology** **BCSE355L : Cloud Architecture Design** **FALL 2026-2027** **Course Project -Review II**

Project Title:	CloudPulse: Autonomous Cloud Reliability and Self-Healing Control Plane Using Real AWS Multi-Service Architecture
Team Name	Nebula Knight

Project Team

S.No	Register Number	Student Name	Signature	Guided By
1	24BIT0246	Moorthy Chetan		

Team Member(s) Contribution and Performance Assessment

Components	Student 1	Student 2	Student 3
Literature Survey (03)			
Design (06)			
Implementation (06)			
Total (15)			
Components			

Expectation for Next Reviews	Comments

Name & Signature of the Evaluator